

APRESENTAÇÃO · SKILLS DO CLAUDE

vercel-react-best-practices

Como guiar o Claude a escrever React e Next.js performáticos por padrão

70 regras · 8 categorias · mantida pela Vercel

Apresentação técnica · ~25 a 30 minutos



Agenda

01

Abertura

Por que skills mudam o jogo no Claude Code

2 min

02

O que é e qual problema resolve

70 regras de performance organizadas por impacto

6 min

03

Como instalar

Um comando, projeto ou global

3 min

04

Como usar, configurar e variar

Gatilho automático, invocação manual, combinação

6 min

05

Demos em vídeo

Waterfall · Bundle · Re-render · antes e depois

8 a 10 min

06

Encerramento

Chamado para ação

1 min

Quem nunca pediu um componente React pro Claude...

...e recebeu um código que funciona, mas segura a página por 3 segundos em produção?

Analogia

Skills são o livro de receitas oficial da casa.

O Claude já sabia cozinhar — o modelo tem talento. A skill ensina ele a cozinhar do jeito do restaurante: usando ingredientes que rendem, técnicas que escalam e evitando os erros que o chef-titular já cansou de ver.

02 · O que é, para que serve, qual problema resolve

Um guia de performance para React e Next.js, mantido pela Vercel

Não é linter. Não é biblioteca.

É um conjunto de instruções que o Claude consulta sozinho antes de escrever, revisar ou refatorar React/Next.

O problema que resolve

Modelos de IA são bons em sintaxe — mas cometem os mesmos erros silenciosos de performance que um dev júnior: waterfalls de await, imports barrel inflando bundle, re-renders desnecessários, ausência de Suspense.

70

regras de performance
prioritizadas por impacto

8

categorias do crítico ao avançado

80%

dos problemas em produção saem
com as 2 primeiras categorias

As 8 categorias por prioridade

Cada regra tem um prefixo. Quanto maior o impacto, mais alto na lista.

● 1	Eliminar waterfalls	CRÍTICO	async-
● 2	Bundle size	CRÍTICO	bundle-
● 3	Performance server-side	ALTO	server-
● 4	Data fetching no cliente	MÉDIO-ALTO	client-
● 5	Re-render	MÉDIO	rerender-
● 6	Performance de renderização	MÉDIO	rendering-
● 7	Performance JavaScript	BAIXO-MÉDIO	js-
● 8	Padrões avançados	BAIXO	advanced-

Um comando. Trinta segundos.



terminal

```
$ npx skills add https://github.com/vercel-labs/agent-skills \
  --skill vercel-react-best-practices
```

1

Abra o projeto

Next.js ou Vite, tanto faz

2

Rode o comando

Instala em `.claude/skills/`

3

Verifique

ls `.claude/skills/`

4

Pronto

Já está ativa no Claude Code

Dica · instale por projeto (vai no `.claude/` versionado no git) — todo o time recebe a mesma skill.

Três modos de uso — você raramente vai "chamar" a skill

A description da skill age como gatilho. O Claude ativa sozinho quando faz sentido.

MODO 1

Automático

o ideal

Você pede um componente que busca dados de usuário e pedidos. O Claude aplica `async-parallel` sozinho. Você só vê o código bom.

```
"crie um componente que busca user e orders"
```

MODO 2

Invocação explícita

para auditoria

Você pede uma revisão usando o nome da skill. Útil para varrer código legado e listar os problemas encontrados.

```
"revise esse arquivo usando vercel-react-best-practices"
```

MODO 3

Combinação

skill stack

Funciona junto com `next-best-practices`, `next-cache-components` e `deploy-to-vercel`. Uma cuida de performance, outra de cache.

```
"use as skills next + react-best-practices"
```

Três pontos que costumam confundir

01

A skill é read-only

Ela guia, não muda código sozinha. Quem escreve continua sendo o Claude — a skill só garante que ele lembre das 70 regras antes de gerar a primeira linha.

02

Você pode customizar o SKILL.md

Edite localmente para relaxar uma regra que não se aplica ao seu stack, ou para adicionar contexto do time. Versione no git e todo mundo herda.

03

AGENTS.md tem todas as 70 regras expandidas

Cada regra com exemplo errado e exemplo certo. Fonte oficial para consulta humana — vale a leitura mesmo fora do Claude Code.



05

Demos em vídeo

Três cenários · mesmo roteiro · antes e depois



Cenário → pedido sem skill → pedido com skill → diff → métrica

Waterfall de fetches no dashboard

Página que precisa de `getUser()`, `getOrders()` e `getNotifications()`

SEM SKILL

```
await getUser()  
await getOrders()  
await getNotifications()
```

~900ms

COM SKILL · async-parallel

```
const [user, orders, notif] =  
  await Promise.all([  
    getUser(),  
    getOrders(),  
    getNotifications()  
  ])
```

~300ms

Impacto: 600ms a menos no Time to Interactive · waterfall do DevTools mostra na hora

Bundle inflado por barrel import

Componente que usa um único método do lodash

SEM SKILL

```
import { debounce } from 'lodash'
```

bundle inclui o lodash inteiro

~70 KB

COM SKILL · bundle-barrel-imports

```
import debounce from 'lodash/debounce'
```

import direto, sem barrel

~2 KB

Impacto: 35× menor · print do next build mostra a redução do chunk

Re-render explosivo por prop não-primitiva

Input pai com useState — filho recebe objeto recriado a cada keystroke

SEM SKILL

Objeto literal nas props.

```
<Filho  
  config={{ x: 1, y: 2 }}  
>
```

30 renders

ao digitar uma palavra

COM SKILL · rerender-memo-with-default-value

Default hoisted, referência estável.

```
const CONFIG = { x:1, y:2 }  
<Filho config={CONFIG} />
```

1 render

o filho só atualiza quando precisa

Impacto: React DevTools Profiler mostra a queda de renders ao vivo

60 a 90 segundos. Mesmo formato nos três.



Exemplo de punchline: "3 segundos pra ler 70 regras, 300ms a menos pro usuário."

Instala hoje. Deixa rodando uma semana.

*Me conta na próxima reunião
quantos waterfalls você não escreveu.*



```
$ npx skills add https://github.com/vercel-labs/agent-skills --skill vercel-react-best-practices
```